

Implementacao 2 - Mandelbrot

Infraestrutura de Software

Identificacao da entrega: bgs

1. Objetivo

O projeto calcula o conjunto de Mandelbrot em uma versao serial e tres versoes paralelas. Foram implementadas uma versao com OpenMP e duas versoes com Pthreads, utilizando divisoes de trabalho distintas. A entrega tambem gera os quatro arquivos de intensidades, registra os tempos de execucao e inclui compilacao automatizada, testes e evidencias de validacao.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidencia
Calculo serial	OK	make test e inspecao de src/mandelbrot.c
OpenMP com numero solicitado de threads	OK	parallel for com num_threads(config->num_threads)
Pthreads 1 - blocos contiguos	OK	Inspecao do particionamento e testes
Pthreads 2 - linhas ciclicas	OK	y = thread_id; y < height; y += num_threads
Mesma matematica e arquivos identicos	OK	cmp entre serial e as tres versoes paralelas
Arquivos sem cabecalho PGM	OK	Inicio do arquivo contem somente intensidades
Tempos referentes principalmente ao calculo	OK	clock_gettime envolve calculo; escrita ocorre depois
Validacao de argumentos e overflow	OK	strtol, positivos e verificacao de overflow
Execucao normal sem stdout	OK	Execucao principal e tests/test.sh
Compilacao sem warnings	OK	make clean && make com -Wall -Wextra -Werror
Valgrind	Parcial	0 definitely lost; 864 bytes possibly lost no caminho libgomp/OpenMP

3. Como Reproduzir

No diretorio raiz do projeto, em Linux/Ubuntu:

```
make clean
make
./mandelbrot 800 600 1000 4
make test
cmp mandelbrot_bgs_serial.pgm mandelbrot_bgs_openmp.pgm
cmp mandelbrot_bgs_serial.pgm mandelbrot_bgs_pthreads1.pgm
cmp mandelbrot_bgs_serial.pgm mandelbrot_bgs_pthreads2.pgm
```

Repositorio GitHub: <https://github.com/BrenoSanti/mandelbrot-infra-software>

4. Arquitetura

- src/main.c - valida argumentos, aloca memoria, mede os calculos e grava as saidas.
- src/mandelbrot.h - define MandelbrotConfig e declara as quatro implementacoes.
- src/mandelbrot.c - contem a matematica compartilhada e as estrategias de execucao.
- tests/test.sh - testa casos validos, saidas identicas e entradas invalidas.
- Makefile - automatiza compilacao, limpeza e testes.
- README.md - documenta uso, dependencias, estrategias e testes.
- evidencias.log - registra a sessao manual de validacao.

Decisao -> buffer linear unico -> **Motivo** -> reduzir memoria e manter indexacao uniforme -> **Consequencia** -> cada resultado e gravado antes do proximo calculo.

Decisao -> uma unica funcao de calculo de pixel -> **Motivo** -> garantir a mesma matematica -> **Consequencia** -> as diferencas ficam restritas a distribuicao das linhas.

Decisao -> medir criacao e sincronizacao das threads, mas nao a escrita -> **Motivo** -> representar o custo computacional -> **Consequencia** -> os tempos Pthreads incluem seu overhead.

5. Estrategias e Diario de Desenvolvimento

5.1 Estrategias

Estrategia	Nome curto	Contexto	Motivo da troca
Percurso sequencial por linhas	Serial	Referencia deterministica	Nao houve troca registrada
parallel for com schedule(static)	OpenMP	Paralelismo por diretiva	Nao houve troca registrada
Blocos contiguos balanceados	Pthreads 1	Divisao explicita	Nao houve troca registrada
Linhas intercaladas	Pthreads 2	Divisao explicita alternativa	Nao houve troca registrada

5.2 Diario

#	O que foi validado	Resultado	Evidencia
1	Compilacao limpa	Sucesso	evidencias.log
2	Execucao 800 x 600, 1000 iteracoes, 4 threads	Exit code 0 e quatro arquivos gerados	evidencias.log
3	Comparacao das quatro saidas	Todas identicas	cmp e Figura 1
4	Testes automatizados	Todos os testes passaram	make test e Figura 1
5	Entradas invalidas	stderr e exit code 1	evidencias.log e Figura 2
6	Caso limite 1 x 1 x 1, 1 thread	Exit 0; quatro saidas identicas	evidencias.log
7	Auditoria dos arquivos fonte	Estrategias confirmadas	Inspecao manual
8	Valgrind 23 x 17, 100 iteracoes, 4 threads	Parcial: 0 definitely lost; 864 possibly lost via libgomp	Saida do Valgrind
9	Git/GitHub	Commit c66b679 e push da main concluidos	Repositorio remoto

6. Evidencias

A sessao foi iniciada com script -a evidencias.log, seguida por date, whoami e pwd. O registro identifica o usuario breno, o diretorio do projeto e a data de 02/09/2026. O log contem compilacao limpa, execucao principal, tempos, comparacoes, make test, entradas invalidas e caso limite.

Tempos observados para 800 x 600, 1000 iteracoes e 4 threads:

Versao	Tempo (s)
Serial	0.288573555
OpenMP	0.155550992
Pthreads 1	0.164276594
Pthreads 2	0.089144066

Os valores correspondem a uma unica medicao observada e nao sustentam, isoladamente, uma conclusao geral de desempenho.

Valgrind: definitely lost = 0 bytes; indirectly lost = 0 bytes; possibly lost = 864 bytes; still reachable = 2.608 bytes; ERROR SUMMARY = 1 contexto. O stack trace do bloco 'possibly lost' passa por pthread_create, libgomp e GOMP_parallel durante mandelbrot_openmp. A verificacao e registrada como parcial.

```
breno@victor: /mnt/c/Users/victor/OneDrive/Área de Trabalho/terraflora/implementacao2/bgs$  
=== VALIDACAO FINAL - MANDELBROT ===  
rm -f mandelbrot src/main.o src/mandelbrot.o mandelbrot_bgs_*.pgm times.txt  
gcc -Wall -Wextra -Werror -std=c11 -O2 -fopenmp -pthread -c src/main.c -o src/main.o  
gcc -Wall -Wextra -Werror -std=c11 -O2 -fopenmp -pthread -c src/mandelbrot.c -o src/mandelbrot.o  
gcc -Wall -Wextra -Werror -std=c11 -O2 -fopenmp -pthread src/main.o src/mandelbrot.o -o mandelbrot -lm  
Serial = OpenMP: OK  
Serial = Pthreads1: OK  
Serial = Pthreads2: OK  
sh tests/test.sh  
Todos os testes passaram.  
=== VALIDACAO FINAL CONCLUIDA ===  
breno@victor: /mnt/c/Users/victor/OneDrive/Área de Trabalho/terraflora/implementacao2/bgs$
```

Figura 1 - Validacao final: compilacao, comparacao das quatro implementacoes e testes automatizados.

```
breno@victor: /mnt/c/Users/victor/OneDrive/Área de Trabalho/terraflora/implementacao2/bgs$  
=== TESTE DE TRATAMENTO DE ERROS ===  
Erro: num_threads deve ser um inteiro positivo valido.  
EXIT CODE: 1  
  
Erro: largura deve ser um inteiro positivo valido.  
EXIT CODE: 1  
  
Uso: ./mandelbrot largura altura max_iteracoes num_threads  
EXIT CODE: 1  
=== VALIDACAO DE ERROS CONCLUIDA ===  
breno@victor: /mnt/c/Users/victor/OneDrive/Área de Trabalho/terraflora/implementacao2/bgs$ |
```

Figura 2 - Tratamento de entradas invalidas: threads, largura e quantidade de argumentos, todos com exit code 1.

7. Uso de IA

Durante o desenvolvimento, utilizei IA como ferramenta de apoio para esclarecer duvidas sobre a implementacao, revisar trechos do codigo, auxiliar na identificacao de possiveis erros e sugerir formas de testar as diferentes versoes do programa.

A IA tambem foi utilizada como apoio na organizacao da documentacao e na revisao dos requisitos do trabalho, incluindo as diferencas entre a implementacao serial, OpenMP e as duas estrategias com Pthreads.

As sugestoes fornecidas foram verificadas por meio da compilacao e execucao do programa, comparacao dos arquivos com cmp, execucao de make test, testes com entradas invalidas e caso limite, inspecao do codigo-fonte e analise com Valgrind. Os resultados finais foram conferidos manualmente antes da entrega.

8. Reflexao Final

A implementacao mostrou que o mesmo calculo pode ser mantido enquanto apenas a distribuicao do trabalho muda. A versao OpenMP simplifica o paralelismo por diretiva, enquanto as versoes Pthreads tornam explicitas a criacao, a divisao e a sincronizacao das threads. A divisao por blocos trabalha com conjuntos contiguos de linhas, enquanto a ciclica espalha as linhas entre as threads. O compartilhamento da funcao de pixel ajudou a manter as quatro saidas identicas. Os testes de entradas invalidas e de caso limite reforçaram a robustez do programa. Uma comparacao de desempenho mais forte exigiria repeticoes e analise estatistica.

9. Checklist Final de Entrega

Item	Status
Compilacao do zero seguindo o relatorio	OK
Testes executados e evidencias.log gerado	OK
Capturas obrigatorias incluidas	OK
Caso limite testado	OK
Casos invalidos testados	OK
Uso de IA documentado como ferramenta de apoio	OK
Relatorio sem placeholders	OK
PDF final gerado	OK
bgs.tar final	Pendente

10. Se eu tivesse mais 2 horas

- Repetiria cada medicao varias vezes e calcularia mediana e dispersao.
- Compararia speedup e eficiencia com diferentes numeros de threads.
- Executaria mais casos com dimensoes nao divisiveis pelo numero de threads.
- Criaria um teste independente com pixels calculados manualmente.
- Executaria AddressSanitizer e UndefinedBehaviorSanitizer.
- Registraria caracteristicas da CPU e carga do sistema durante as medicoes.